



计算机图形学与虚拟现实

冼楚华

Email: chhxian@scut.edu.cn

华南理工大学计算机科学与工程学院



课程信息

- 授课老师姓名：冼楚华
- Email: chhxian@scut.edu.cn 
- 个人主页: <https://chuhuaxian.github.io/>
- QQ: 89071086 （比较少用，非急事请不要私聊）
- 办公室: B3-202-2
- 课程QQ群（见二维码）



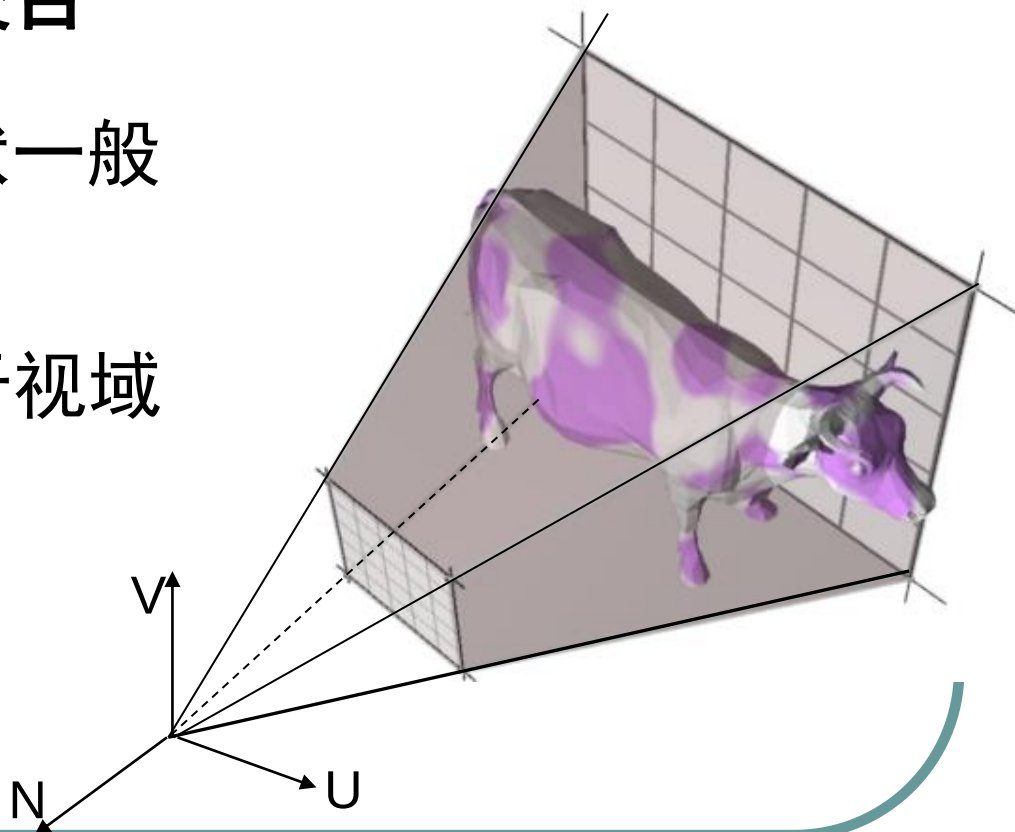
群名称: 2025计算机图形学@SCUT
群 号: 134660465

内容

- **二维线段裁剪(Line Clipping)**
- **二维多边形裁剪 (Polygon Clipping)**
- **三维裁剪(3D Polygon Clipping)**

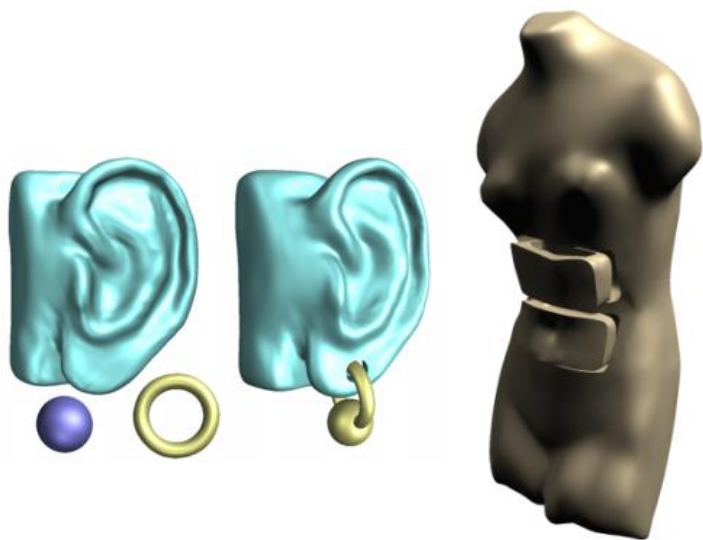
为什么需要裁剪---剔除不可见部分

- 透视投影中视域四棱锥是指位于“前面”和“后面”之间的**四棱台**
- 平行投影的视域形状一般为**长方体**
- 投影时，要剔除位于视域体外部的物体部分。

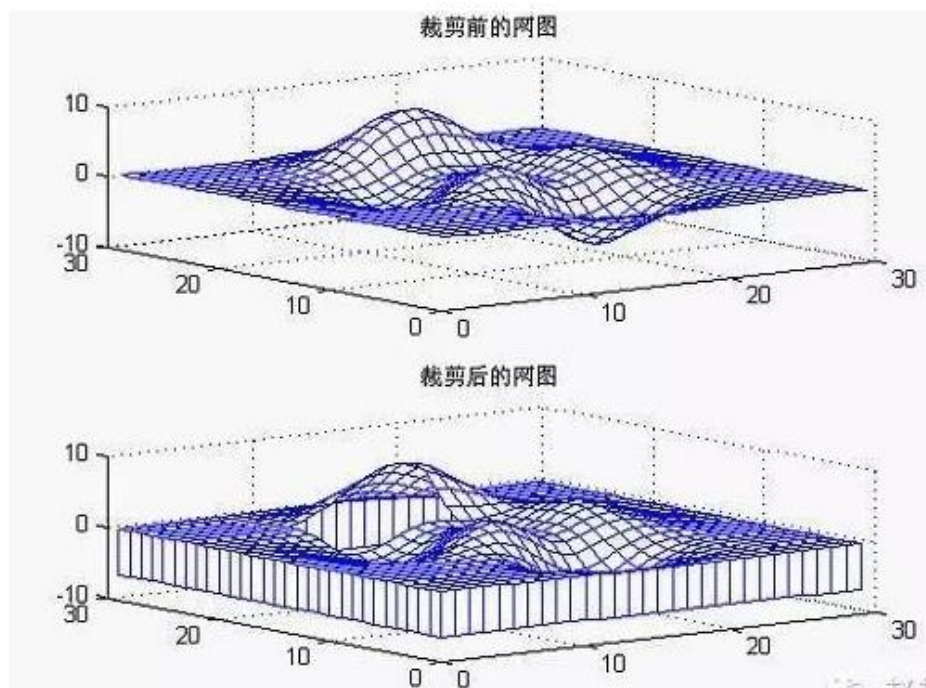


为什么需要裁剪---实体造型

- 实体造型



图形反走样、消隐

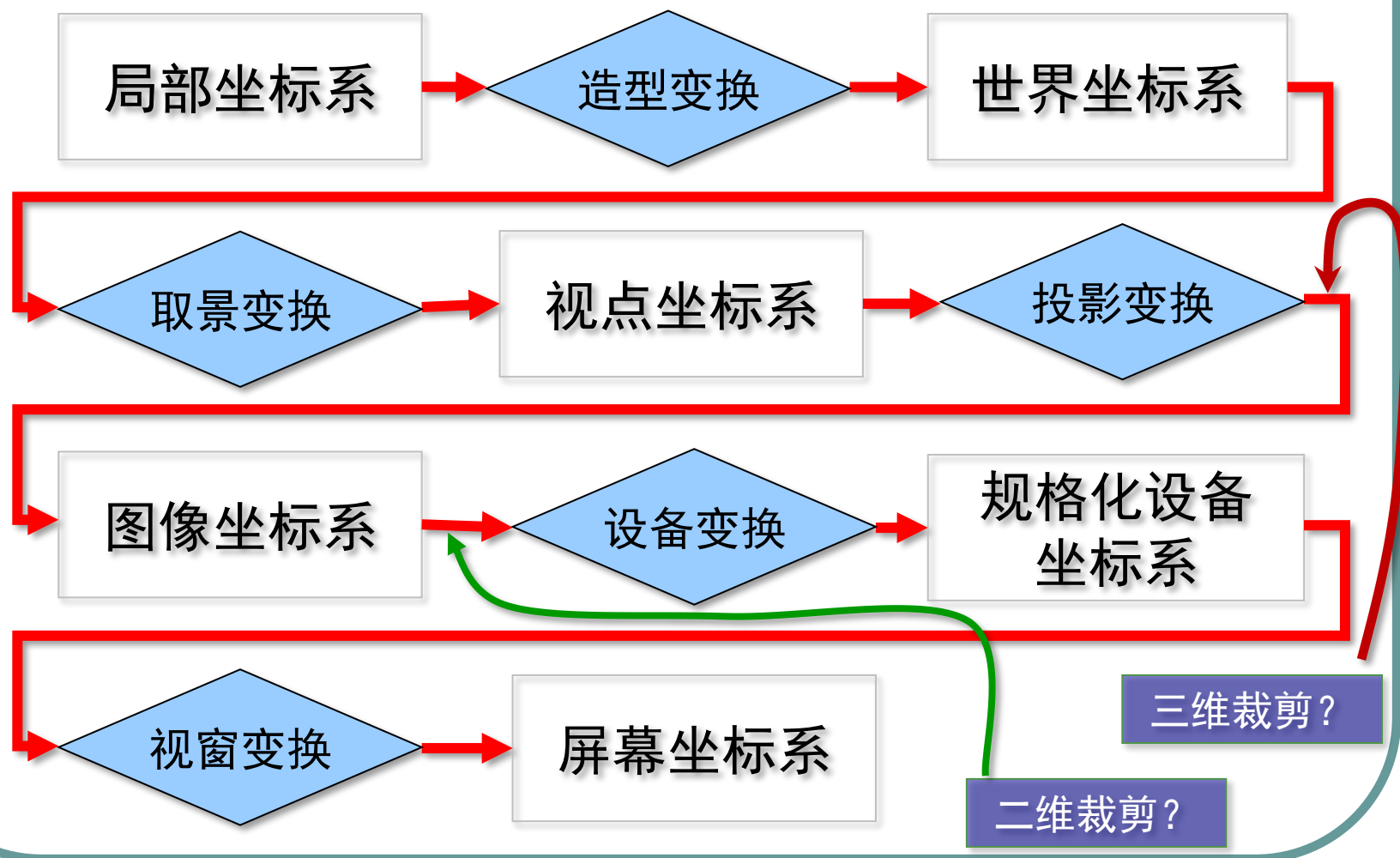


多面体裁剪：实体造型布尔运算

内容 (Section 2.5)

- 二维变换
- 三维变换
- 裁剪
 - 二维线段裁剪
 - 二维多边形裁剪
 - 三维裁剪
 - 关于三维变换与裁剪

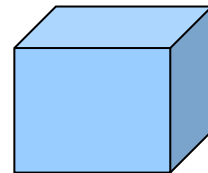
三维变换流程图



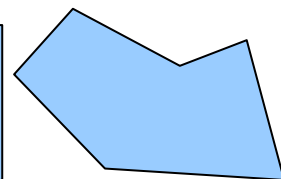
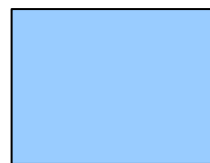
裁剪

- 裁剪算法分类：

- 裁剪窗口维数：二维、三维



- 裁剪窗口类型：规则(矩形、六面体)和不规则的(任意多边形和多面体)



- 裁剪对象类型：点、线、多边形、多面体



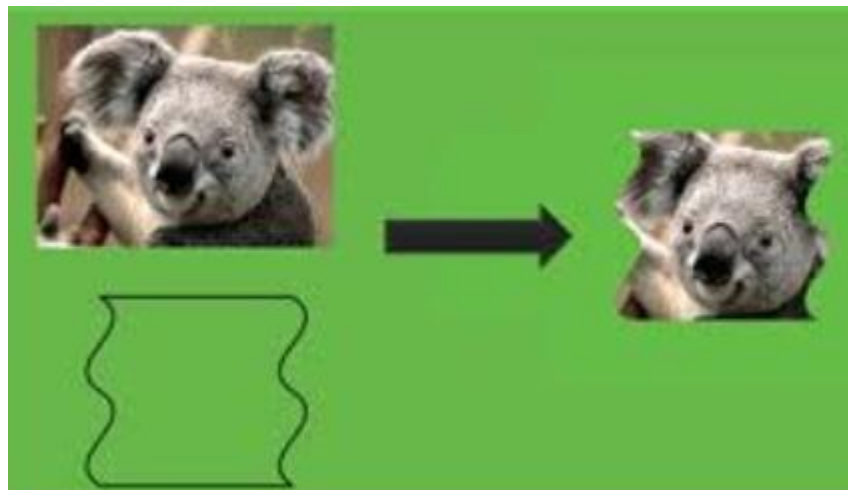
- 实现方式：软件和硬件实现



1 二维线裁剪

● 图形裁剪

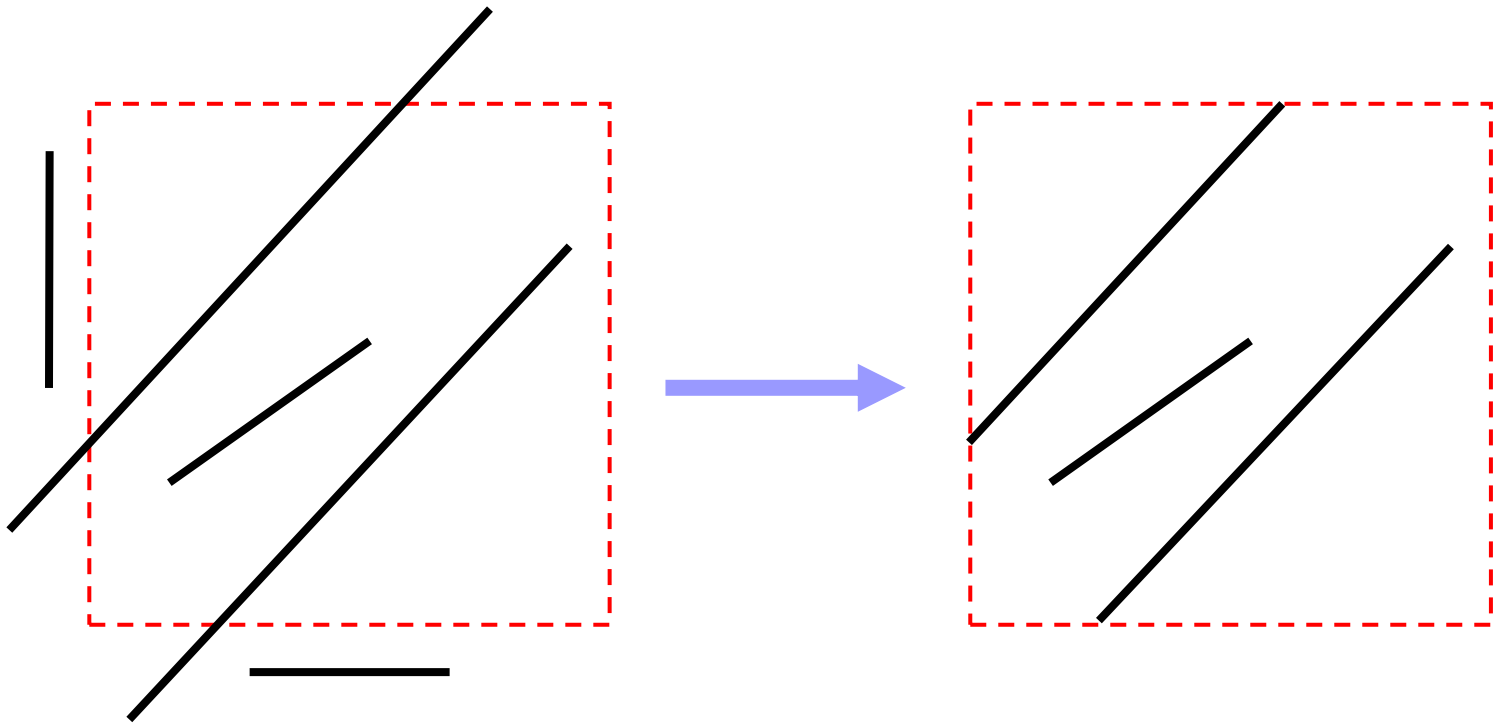
- 确定画面中哪些点、线段或部分线段在裁剪窗口内。
- 显示位于窗口内的，
丢弃其它的。



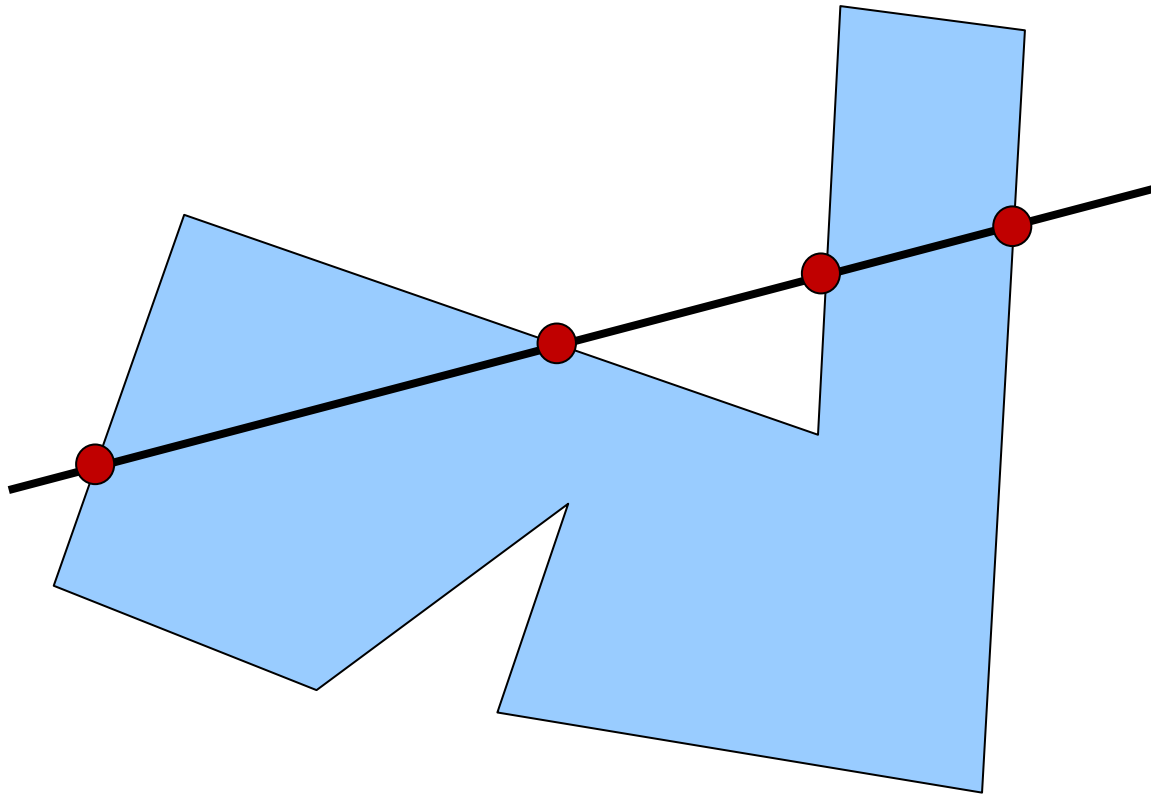
● 裁剪效率

- 需要对大量的点、线段进行裁剪，因此裁剪算法的效率十分重要
- 尽可能快速拒绝和接受，尽量少求交点

二维线裁剪示意



一个简单的方法

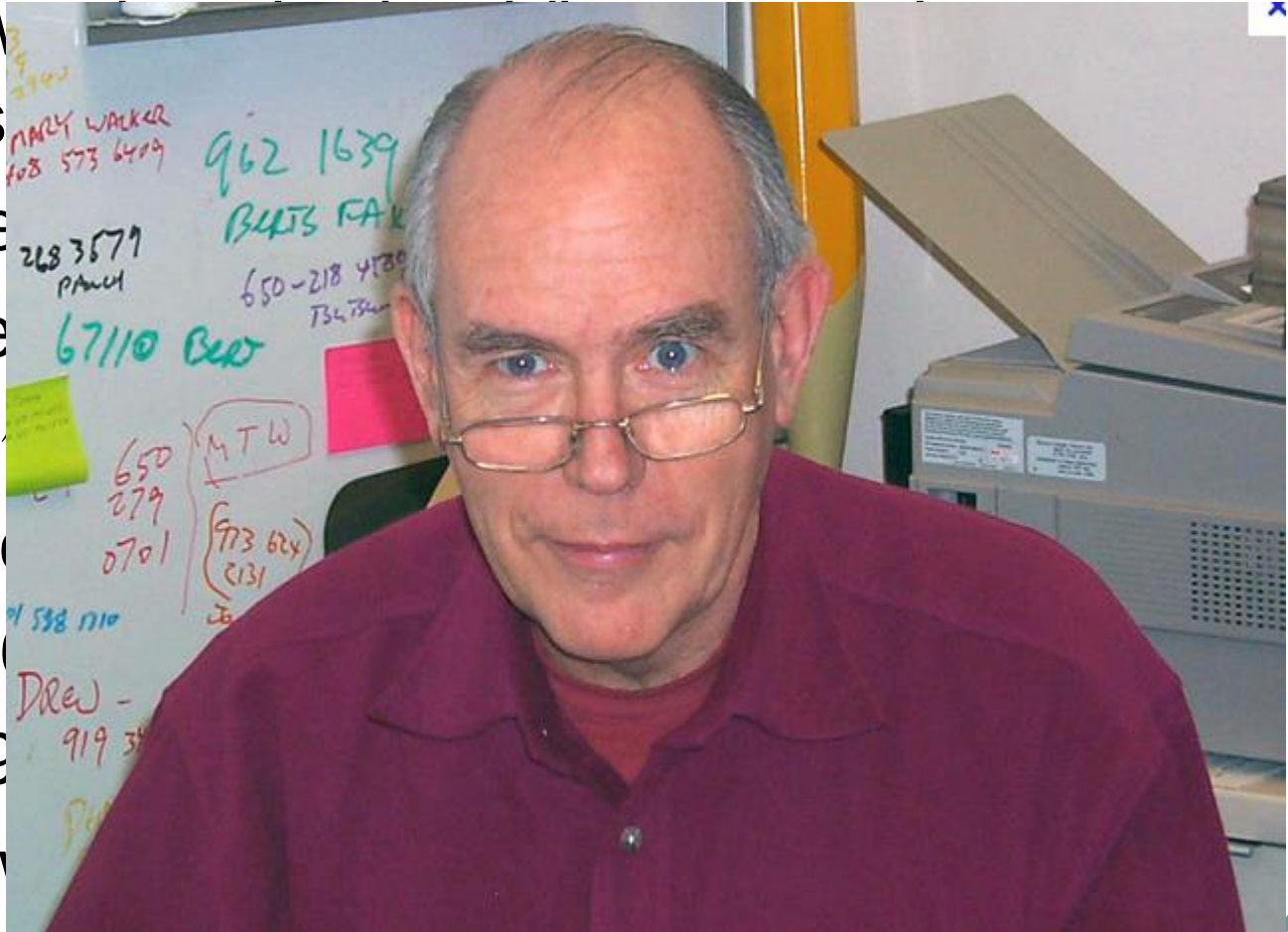


二维线裁剪主要方法

- Sutherland-Cohen 裁剪：编码
- 中点分割裁剪：除以2，移位运算
- 参数化裁剪与梁友栋-Barsky 裁剪：高效率的裁剪
- Nicholl-Lee-Nicholl裁剪：更为精细的判断
-

I. E. Sutherland

- Ivan Edw
- Hastings
- Carnegie
- MIT: Ske
- 1964: 入
- Asso. Pro
- Prof., 19
- Dean, 19
- Turing A



In Utah

- 图灵奖得主**Alan Kay**, (oriented object programming, graphical user interface)
- z缓冲等技术的发明者**Edwin Catmull**
- 提出了反走样技术的**Frank Crow**
- 开发了**Warnock**算法的John Warnock,
- 发明Gouraud着色技术的Henri **Gouraud**
- 几何流水线之父**James H. Clark**等。
- Phong光照, **B.T. Phong**

Companies

- 犹他大学期间**创办**：Evans & Sutherland, 现在仍在提供数字影院

- 发明虚拟现实头盔



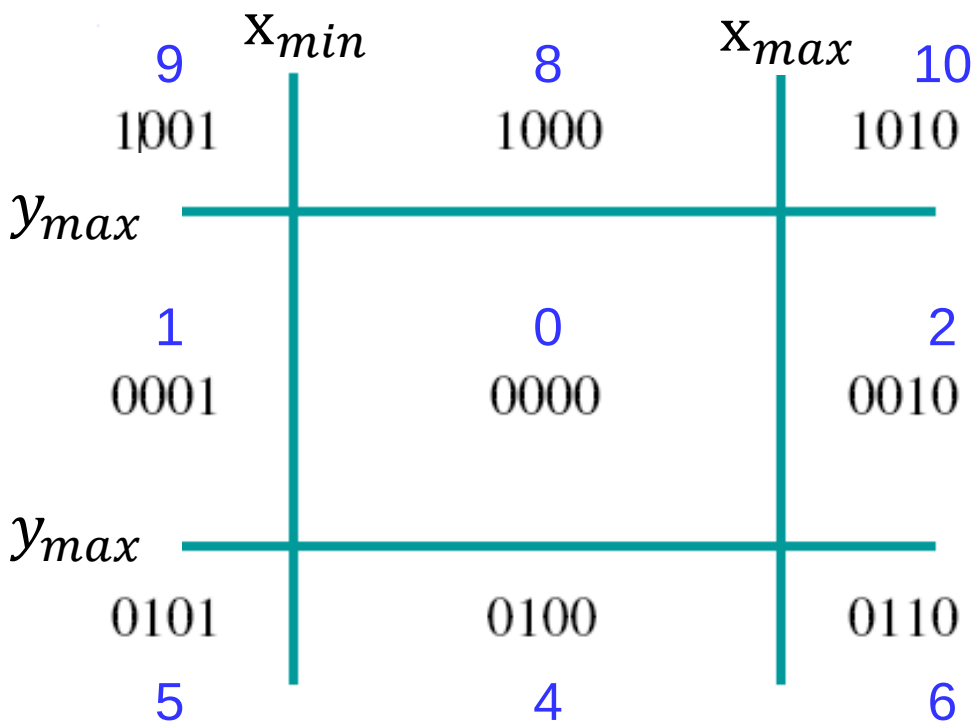
- Caltech：1980
Sutherland, Sproull and Associates 咨询公司

- 被SUN收购
- VLSI

Cohen-Sutherland 线段裁剪 (2.5)

- 裁剪窗口为 **长方形**，裁剪对象为 **线段**
- 裁剪平面被裁剪窗口分成 9 个区域
- 每个区域编码
如图所示.
(outcode)

How to encode?



Outcodes计算

已知窗口四条边的方程分别为

$$y = Y_{max}; x = X_{min}; y = Y_{min}; x = X_{max};$$

```
int ComputeOutode(float x, float y){
```

```
    int code = 0;
```

```
    if y >  $Y_{max}$  then code = 8;
```

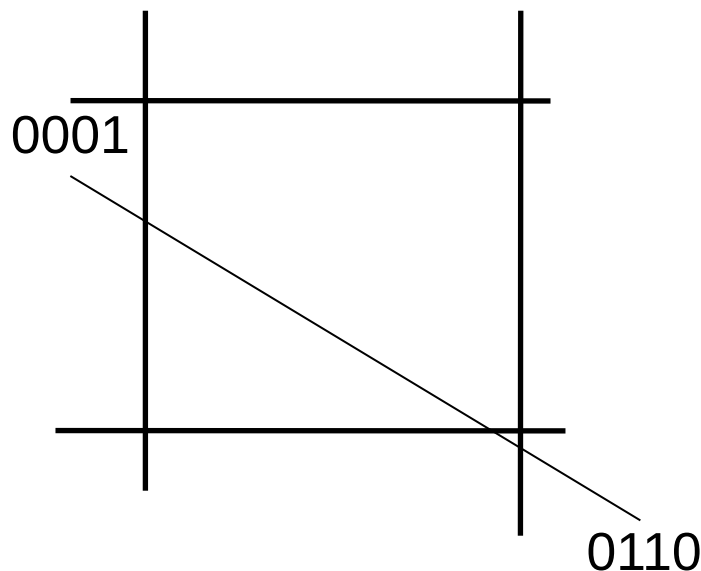
```
    else if y <  $Y_{min}$ ; code = 4;
```

```
    if x >  $X_{max}$  code = code + 2;
```

```
    else if x <  $X_{min}$  = code + 1;
```

```
    return code;
```

```
}
```



裁剪算法(Clipping algorithm)

1. $\text{code1} = \text{ComputeOutCode}(p1), \text{code2} = \text{ComputeOutCode}(p2);$

2. if $\text{code1} \ \& \ \text{code2} \neq 0$ 则位于窗口外

3. else if $\text{code1} = 0 \ \&\& \ \text{code2} = 0$ 则位于窗口内

4. else {

找到两端点中位于窗口外的一个端点p(设为p1);

依次判断p1位于下面哪条直线外侧

① $y = Y_{max}$, ② $x = X_{min}$, ③ $y = Y_{min}$, ④ $x = X_{max}$,

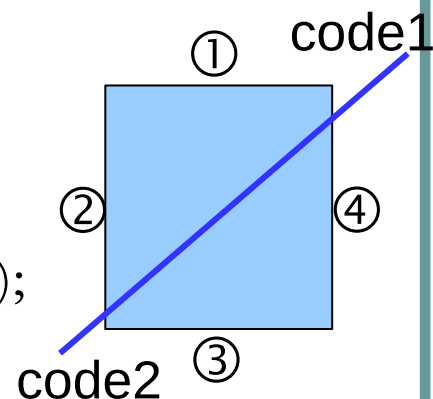
有一个成立则做{

与外侧直线求交点c, 并把cp段丢掉, $p1 = c$;

计算交点p1的outcode;

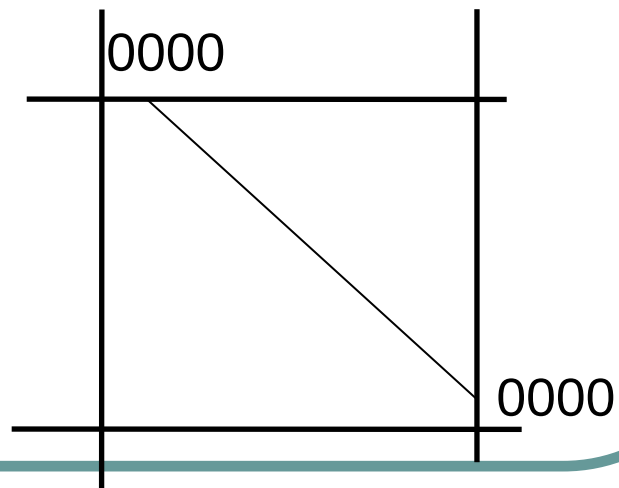
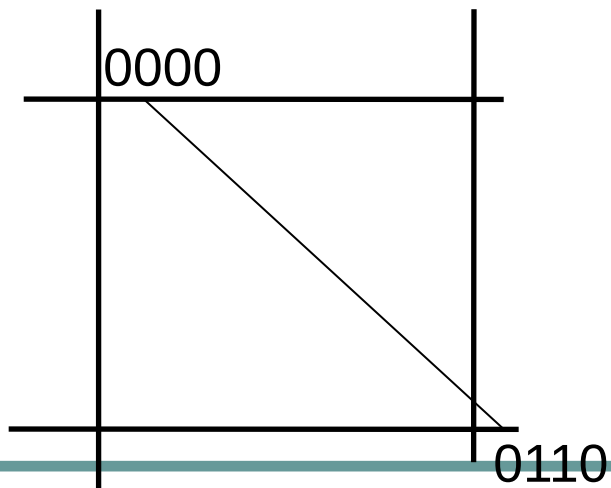
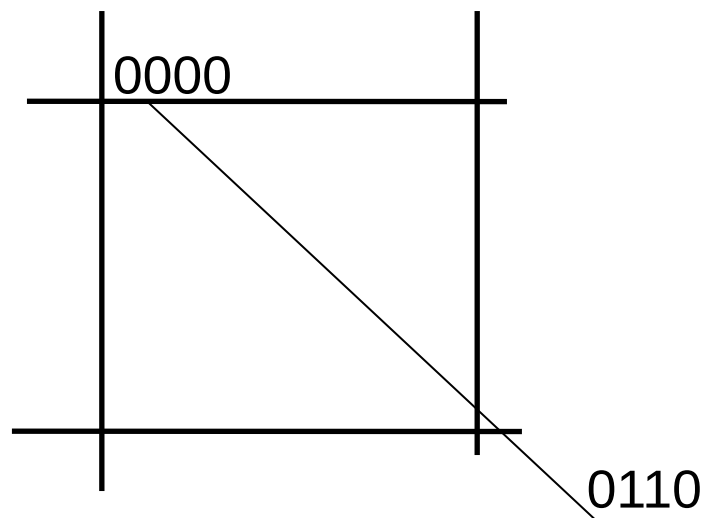
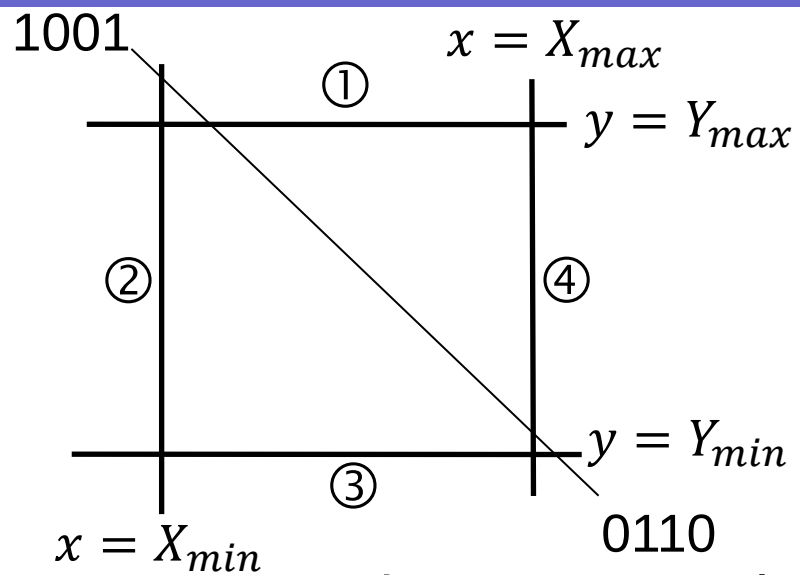
}

}

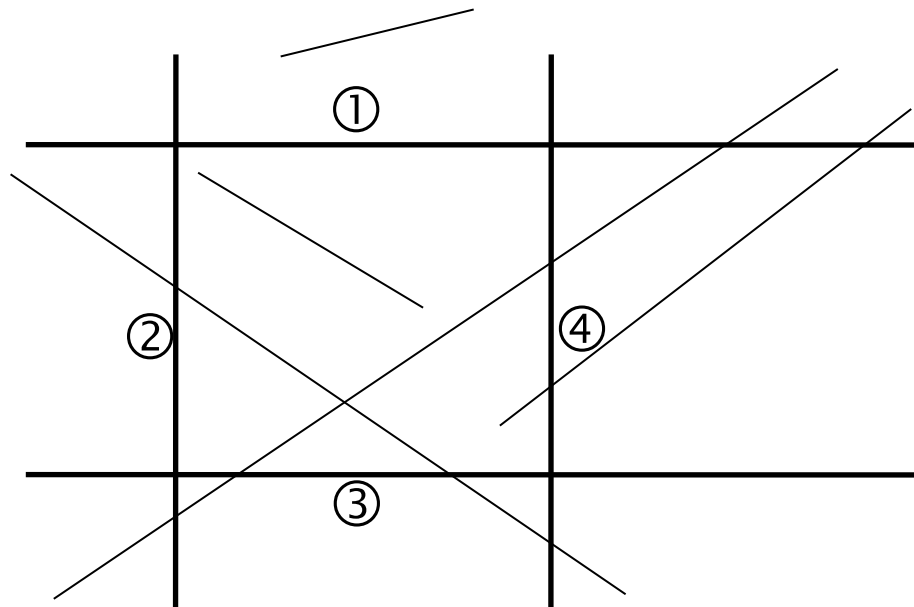


5. 对c到另一端点间的线段重复上述过程直到接受或拒绝;

例



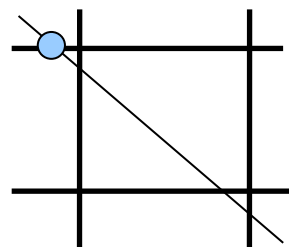
例



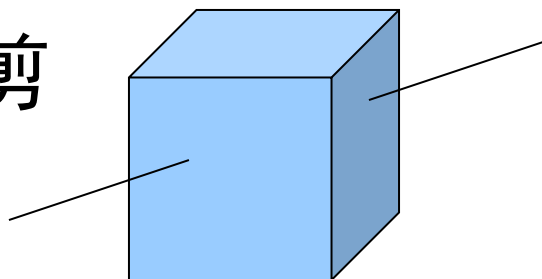
Cohen-Sutherland的改进与扩展

- C-S算法依固定次序对裁剪窗口的边进行相交测试，有可能会求得窗口外的交点。
Nicholl等提出了改进：

Nicholl et al. An efficient new algorithm for 2-D line clipping: its development and analysis.
Siggraph 1987, 253-262.



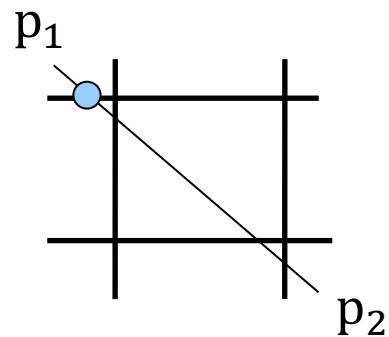
- 易于推广到三维裁剪



梁-Barsky线段裁剪算法

- ACM Transactions on Graphics, 1984 (国内第一篇TOG论文)
- 列入计算机图形学教科书唯一一个中国人提出的算法
- 直线段的参数化表示
 - $x = x_0 + t\Delta x$
 - $y = y_0 + t\Delta y$
 - 点位于矩形内:

$$\begin{cases} x_{min} \leq x_0 + t\Delta x < x_{max} \\ y_{min} \leq y_0 + t\Delta y < y_{max} \end{cases}$$



- 于是有

$$\begin{cases} x_{\min} \leq x_0 + t\Delta x < x_{\max} \\ y_{\min} \leq y_0 + t\Delta y < y_{\max} \end{cases}$$

$$\begin{cases} -\Delta x \cdot t \leq x_0 - x_{\min} \\ \Delta x \cdot t \leq x_{\max} - x_0 \\ -\Delta y \cdot t \leq y_0 - y_{\min} \\ \Delta y \cdot t \leq y_{\max} - y_0 \end{cases}$$

- 上述条件用参数表示为

$$tp_k \leq q_k (k = 1, 2, 3, 4)$$

其中：

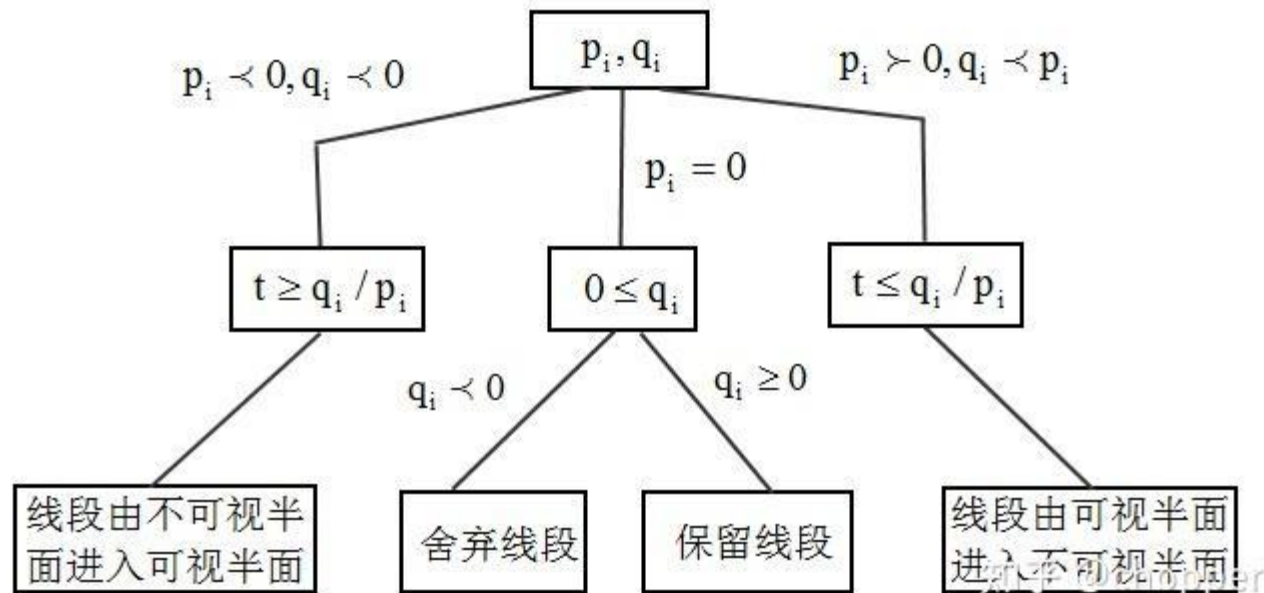
$$p_1 = -\Delta x; \quad q_1 = x_1 - x_{\min}$$

$$p_2 = \Delta x; \quad q_1 = x_{\max} - x_1$$

$$p_3 = -\Delta y; \quad q_1 = y_1 - y_{\min}$$

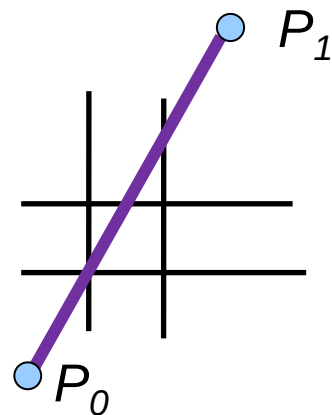
$$p_4 = \Delta y; \quad q_4 = y_{\max} - y_1$$

$$\begin{aligned}
 p_1 &= -\Delta x; & q_1 &= x_1 - x_{min} \\
 p_2 &= \Delta x; & q_1 &= x_{max} - x_1 \\
 p_3 &= -\Delta y; & q_1 &= y_1 - y_{min} \\
 p_4 &= \Delta y; & q_4 &= y_{max} - y_1
 \end{aligned}$$



举个例子来解释Liang-Barsky线段裁剪算法，设线段 P_0P_1 的两个端点分别是 $P_0(-1, -2)$ 和 $P_1(2, 4)$ ，矩形的边界分别是 $x_{\min} = 0, x_{\max} = 1, y_{\min} = 0$ 和 $y_{\max} = 1$ ，则有 $\Delta x = 3$ 和 $\Delta y = 6$ 。等式式 (6)，得到

$$\begin{aligned} p_1 &= -\Delta x = -3, q_1 = x_0 - x_{\min} = -1, & q_1/p_1 &= 1/3; \\ p_2 &= \Delta x = 3, q_2 = x_{\max} - x_0 = 2, & q_2/p_2 &= 2/3; \\ p_3 &= -\Delta y = -6, q_3 = y_0 - y_{\min} = -2, & q_3/p_3 &= 1/3; \\ p_4 &= \Delta y = 6, q_4 = y_{\max} - y_0 = 3, & q_4/p_4 &= 1/2; \end{aligned}$$



根据这些数值，可以计算出 t_0 和 t_1 的值，即

$$\begin{cases} t_0 = \max(\{q_i/p_i | p_i < 0, i = 1, 2, 3, 4\} \cup \{0\}) = \max(1/3, 1/3, 0) = 1/3 \\ t_1 = \min(\{q_i/p_i | p_i > 0, i = 1, 2, 3, 4\} \cup \{1\}) = \min(2/3, 1/2, 1) = 1/2 \end{cases}$$

在实现中特别注意，当 $p_i < 0, q_i \geq 0$ 或者 $p_i > 0, q_i \geq p_i$ 时，可以忽略 q_i/p_i 的计算。

2 二维多边形裁剪

- 裁剪窗口和被裁剪对象：**多边形**

- 对每条边作线裁剪：**线裁剪**

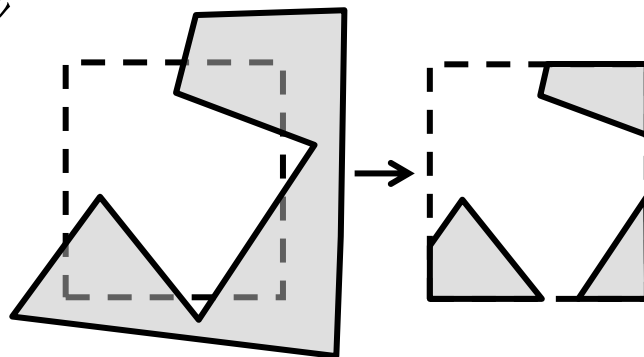
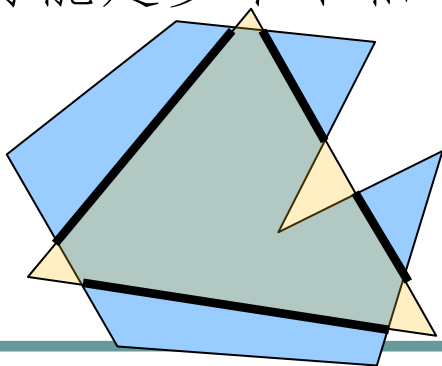
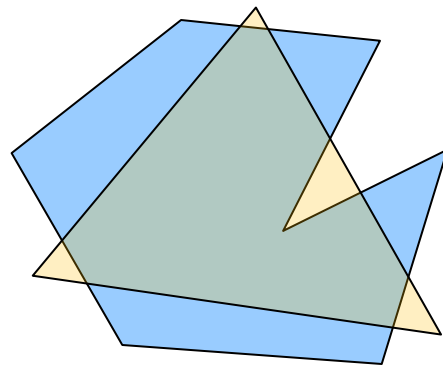
 - 适合线框图, 不适用于多边形着色

- 正确的多边形裁剪：**区域裁剪**

 - 结果仍为封闭多边形

 - 可能会并入一部分窗口作为多边形边界

 - 可能是多个不相连的多边形



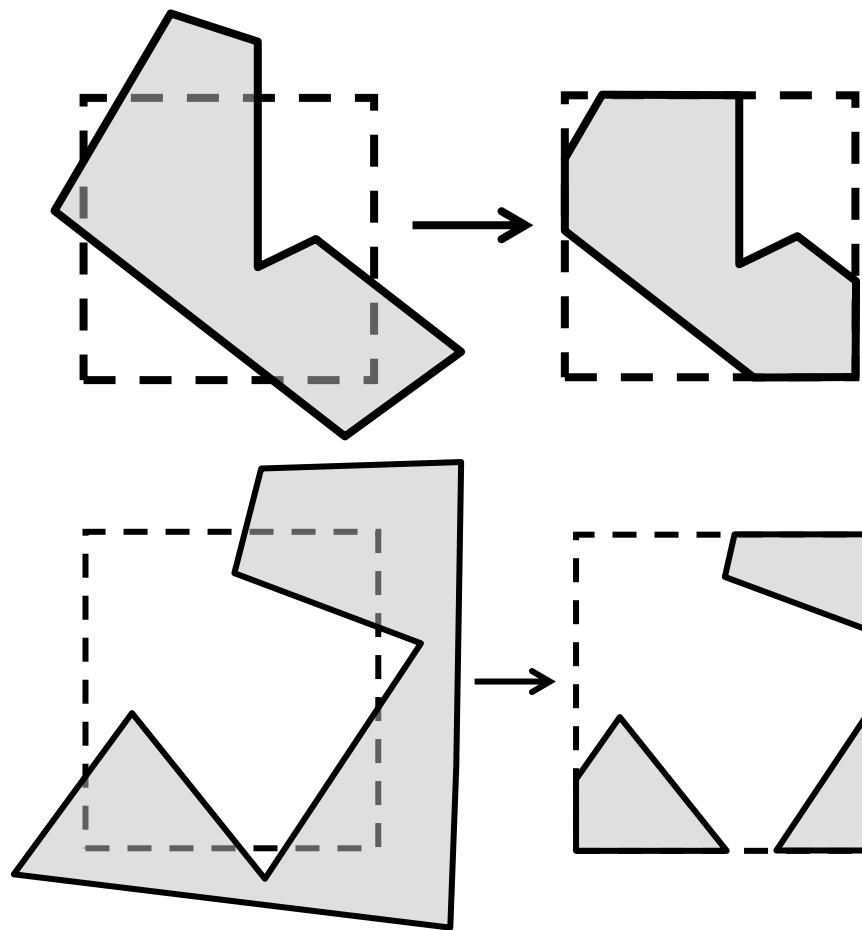
二维多边形裁剪问题描述

● 输入

- 裁剪窗口顶点坐标序列 $\{(u_1, v_1), \dots, (u_n, v_n)\}$
- 被裁多边形顶点坐标序列 $\{(x_1, y_1), \dots, (x_m, y_m)\}$

● 输出

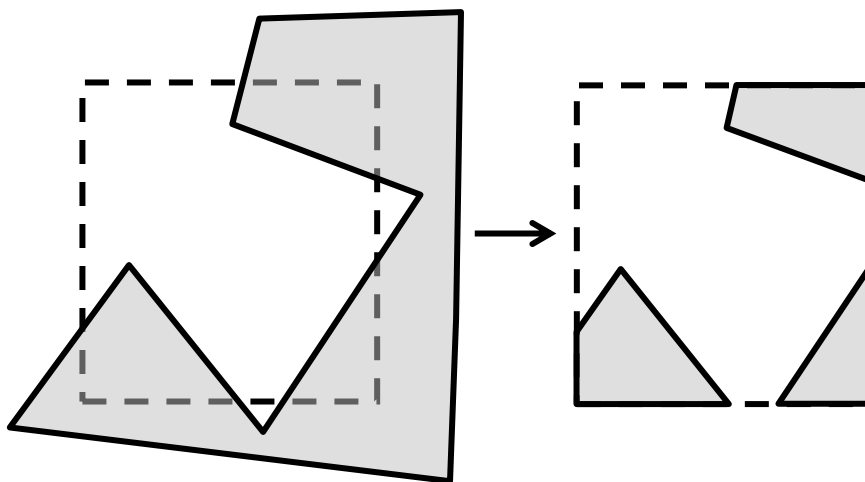
- 按顺序输出裁剪结果的顶点序列
- 保证多边形区域的封闭性



Sutherland-Hodgman多边形裁剪 (Section 8.5)

- 裁剪多边形 (裁剪窗口)：凸多边形
- 被裁剪多边形：任意简单多边形

(I. Sutherland and G. Hodgman. Reentrant polygon clipping. Communication of ACM, 1974, 17(1):32-42.)



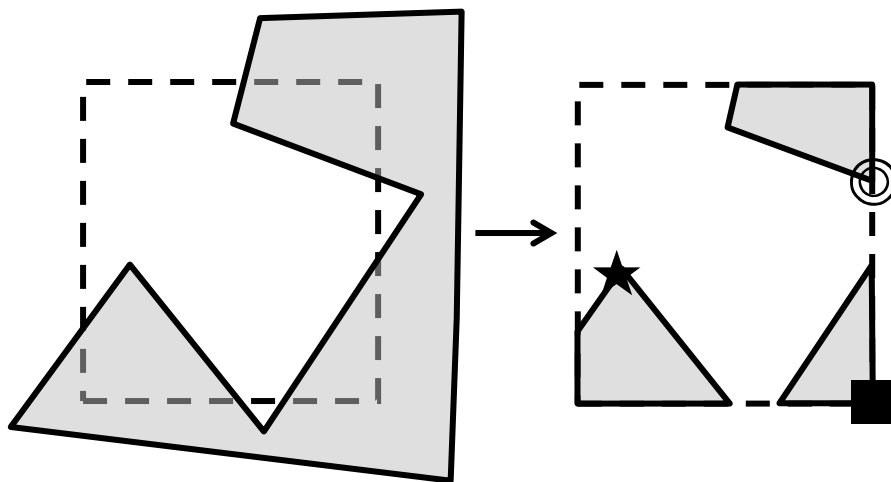
Sutherland-Hodgman 多边形裁剪

● 结果多边形顶点分类

- 被裁剪多边形的顶点★
- 裁剪窗口的顶点■
- 两多边形的交点◎

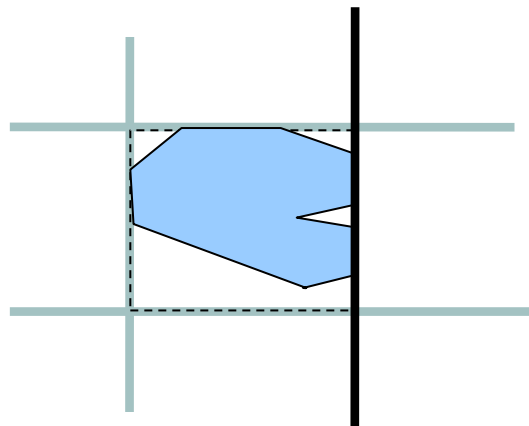
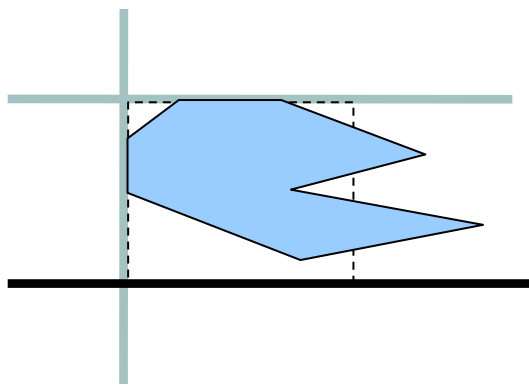
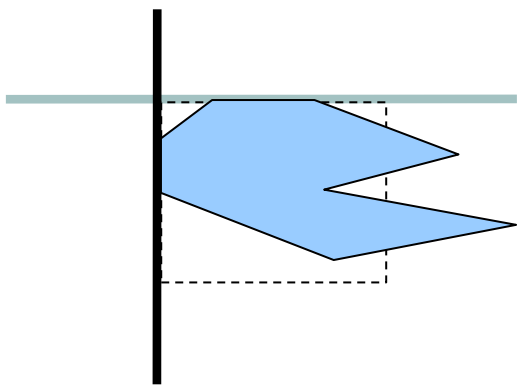
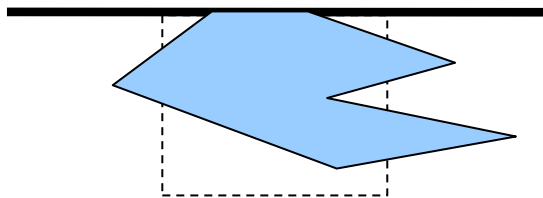
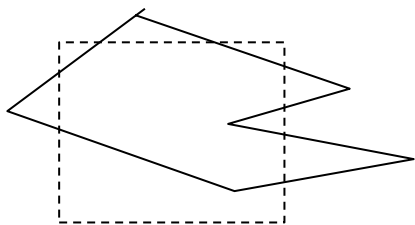
● 结果多边形边分类

- 被裁剪多边形的边(部分)
- 裁剪窗口的边(部分)



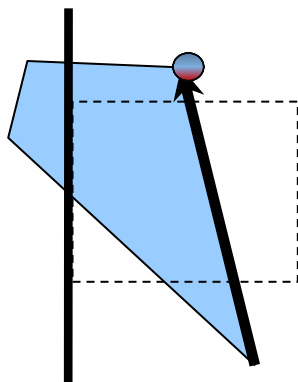
Sutherland-Hodgmont算法

- 逐条取裁剪窗口的边对待裁多边形进行裁剪

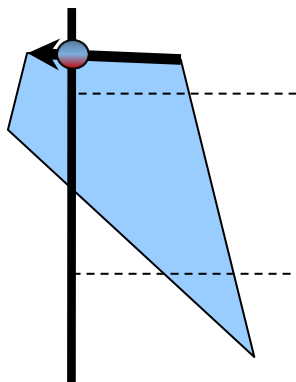


每条边情况

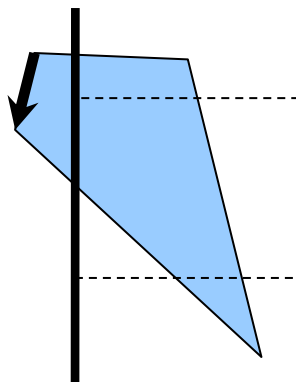
- 输入：上一条边得到的新的被裁剪多边形
- 输出：新的多边形
- 方法：按顺序取出被裁剪多边形的边进行裁剪。共4种可能情况，产生0，1或2个新顶点。



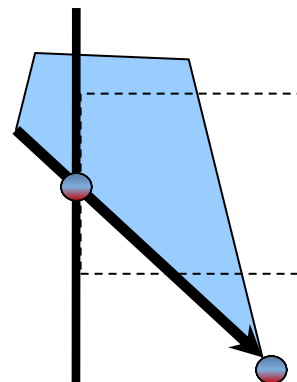
1个



1个



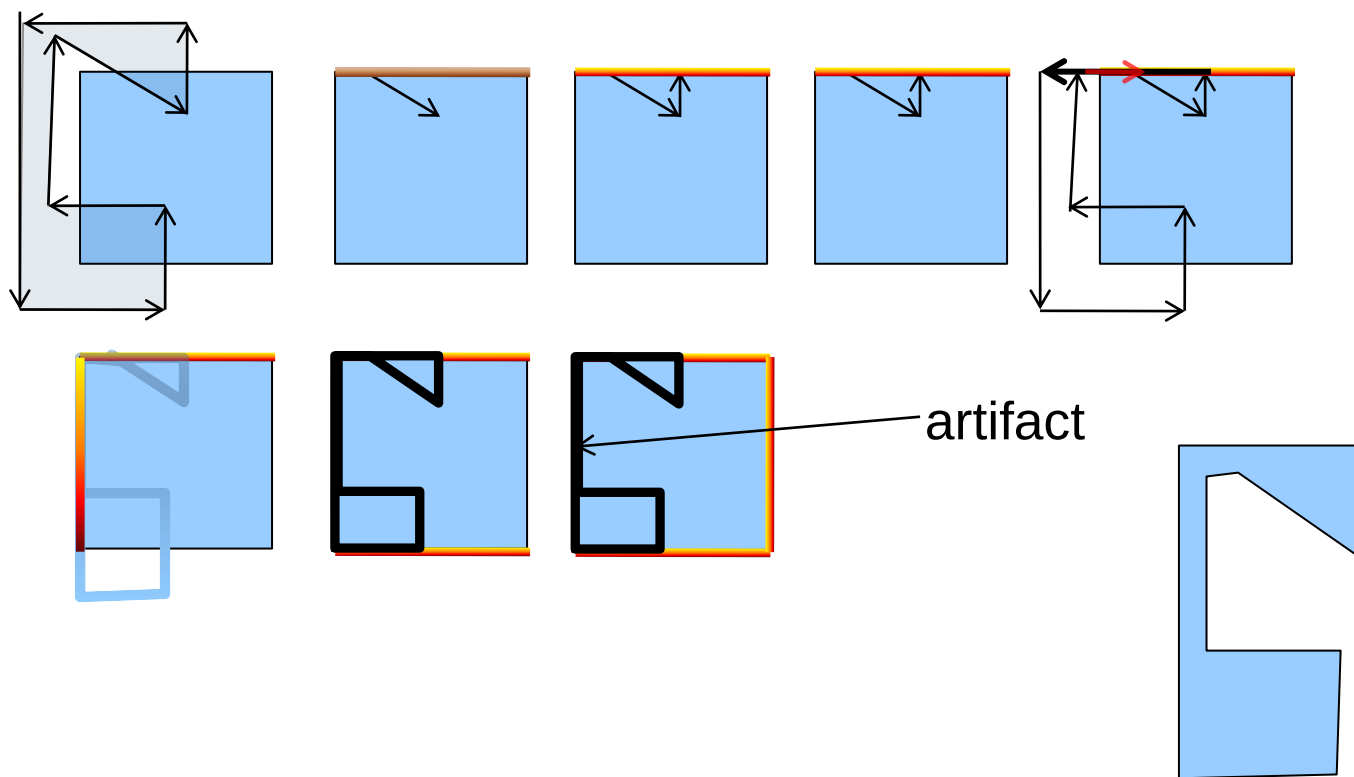
0个



2个

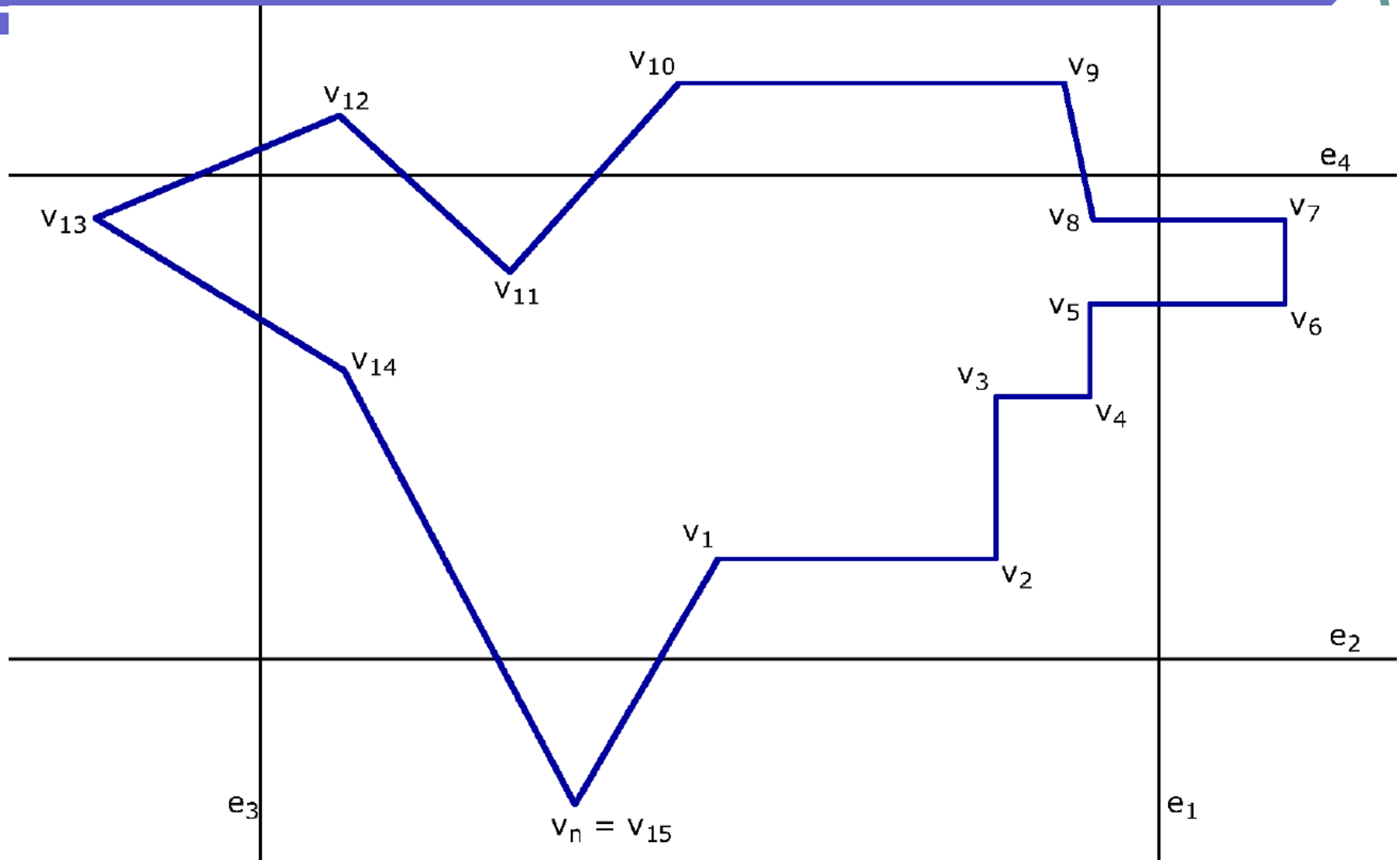
缺点与改进

- 可能包含非多边形内部的窗口边界：需后处理



Example:

http://public.rz.fh-wolfenbuettel.de/~ludwiga/cg/polygonclipping_example.pdf



1st run: Clip edge: e_1 ; inputlist = $\{v_1, v_2, \dots, v_{14}, v_{15}\}$, the given polygon:

from	to	First point	Second point	Case	Add to output-list	remark
v_{15}	v_1	inside	inside	1	v_1	$n = 15$, edge e_1 !
v_1	v_2	inside	inside	1	v_2	
v_2	v_3	inside	inside	1	v_3	
v_3	v_4	inside	inside	1	v_4	
v_4	v_5	inside	inside	1	v_5	
v_5	v_6	inside	outside	2	i_1	
v_6	v_7	outside	outside	3		
v_7	v_8	outside	inside	4	i_2, v_8	
v_8	v_9	inside	inside	1	v_9	
v_9	v_{10}	inside	inside	1	v_{10}	
v_{10}	v_{11}	inside	inside	1	v_{11}	
v_{11}	v_{12}	inside	inside	1	v_{12}	
v_{12}	v_{13}	inside	inside	1	v_{13}	
v_{13}	v_{14}	inside	inside	1	v_{14}	
v_{14}	v_{15}	inside	inside	1	v_{15}	

Output list after first clipping: $o_1 = \{v_1, v_2, v_3, v_4, v_5, i_1, i_2, v_8, v_9, v_{10}, v_{11}, v_{12}, v_{13}, v_{14}, v_{15}\}$

2nd run: Clip edge: e₂; inputlist = o₁:

from	to	First point	Second point	Case	Add to output-list	remark
v ₁₅	v ₁	outside	inside	4	i ₃ , v ₁	v _n = v ₁₅ , edge e ₂ !
v ₁	v ₂	inside	inside	1	v ₂	
v ₂	v ₃	inside	inside	1	v ₃	
v ₃	v ₄	inside	inside	1	v ₄	
v ₄	v ₅	inside	inside	1	v ₅	
v ₅	i ₁	inside	inside	1	i ₁	
i ₁	i ₂	inside	inside	1	i ₂	
i ₂	v ₈	inside	inside	4	v ₈	
v ₈	v ₉	inside	inside	1	v ₉	
v ₉	v ₁₀	inside	inside	1	v ₁₀	
v ₁₀	v ₁₁	inside	inside	1	v ₁₁	
v ₁₁	v ₁₂	inside	inside	1	v ₁₂	
v ₁₂	v ₁₃	inside	inside	1	v ₁₃	
v ₁₃	v ₁₄	inside	inside	1	v ₁₄	
v ₁₄	v ₁₅	inside	outside	2	i ₄	

Output list after second clipping: o₂={i₃, v₁, v₂, v₃, v₄, v₅, i₁, i₂, v₈, v₉, v₁₀, v₁₁, v₁₂, v₁₃, v₁₄, i₄}

3rd run: Clip edge: e_3 ; inputlist = o_2 :

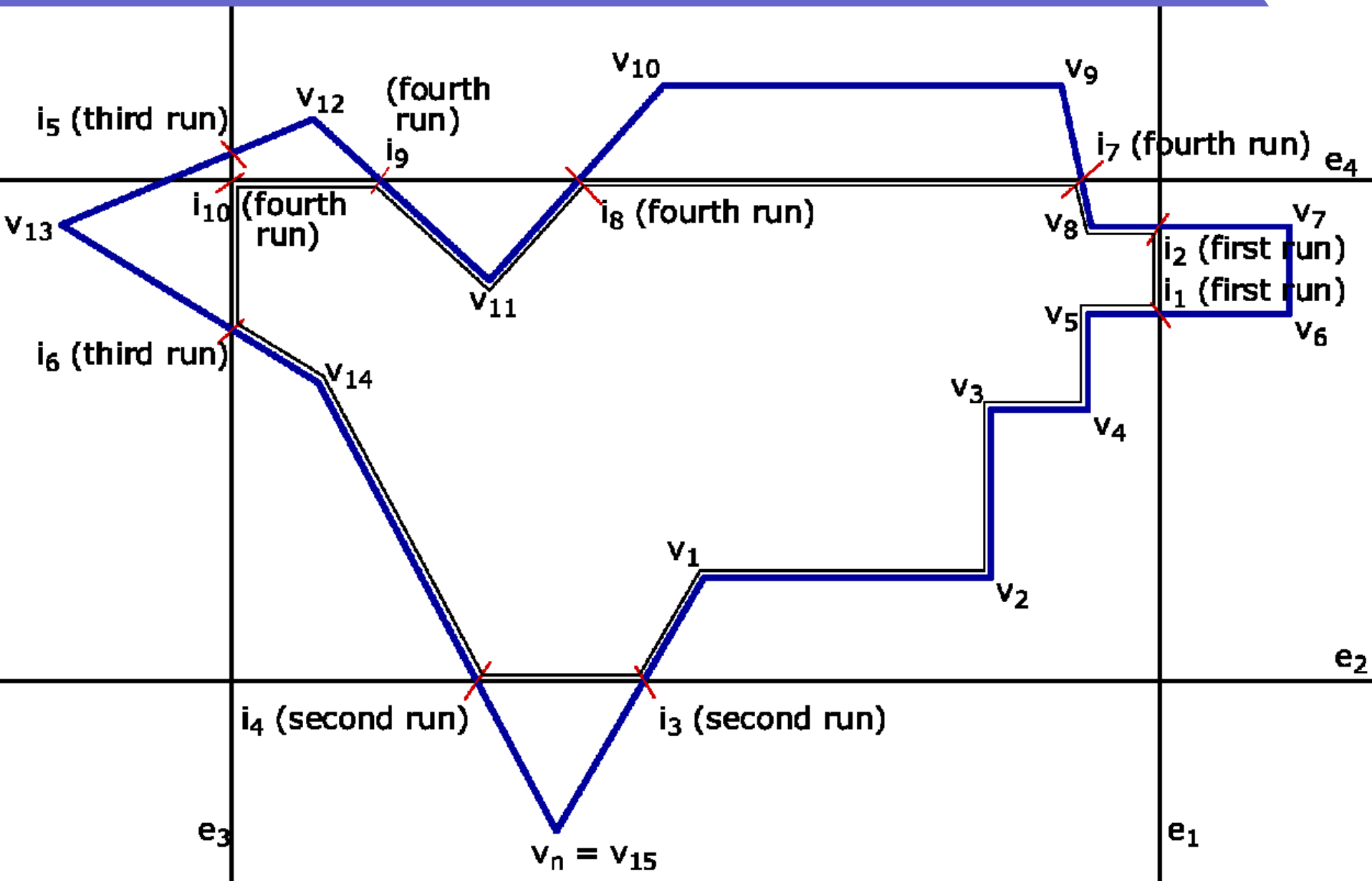
from	to	First point	Second point	Case	Add to output-list	remark
i_4	i_3	inside	inside	1	i_3	$v_n = i_4$; edge e_3 !
i_3	v_1	inside	inside	1	v_1	
v_1	v_2	inside	inside	1	v_2	
v_2	v_3	inside	inside	1	v_3	
v_3	v_4	inside	inside	1	v_4	
v_4	v_5	inside	inside	1	v_5	
v_5	i_1	inside	inside	1	i_1	
i_1	i_2	inside	inside	1	i_2	
i_2	v_8	inside	inside	1	v_8	
v_8	v_9	inside	inside	1	v_9	
v_9	v_{10}	inside	inside	1	v_{10}	
v_{10}	v_{11}	inside	inside	1	v_{11}	
v_{11}	v_{12}	inside	inside	1	v_{12}	
v_{12}	v_{13}	inside	outside	2	i_5	
v_{13}	v_{14}	outside	inside	4	i_6, v_{14}	
v_{14}	i_4	inside	inside	1	i_4	

Output list after third clipping: $o_3 = \{i_3, v_1, v_2, v_3, v_4, v_5, i_1, i_2, v_8, v_9, v_{10}, v_{11}, v_{12}, i_5, i_6, v_{14}, i_4\}$

4th run: Clip edge: e4; inputlist = o3:

from	to	First point	Second point	Case	Add to output-list	remark
i4	i3	inside	inside	1	i3	vn=i4; edge e4!
i3	v1	inside	inside	1	v1	
v1	v2	inside	inside	1	v2	
v2	v3	inside	inside	1	v3	
v3	v4	inside	inside	1	v4	
v4	v5	inside	inside	1	v5	
v5	i1	inside	inside	1	i1	
i1	i2	inside	inside	1	i2	
i2	v8	inside	inside	1	v8	
v8	v9	inside	outside	2	i7	
v9	v10	outside	outside	3		
v10	v11	outside	inside	4	i8, v11	
v11	v12	inside	outside	2	i9	
v12	i5	outside	outside	3		
i5	i6	outside	inside	4	i10, i6	
i6	v14	inside	inside	1	v14	
v14	i4	inside	inside	1	i4	

Output list after fourth clipping: o4={i3, v1, v2, v3, v4, v5, i1, i2, v8, i7, i8, v11, i9, i10, i6, v14, i4}



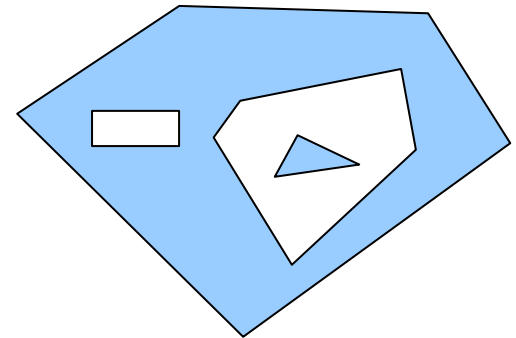
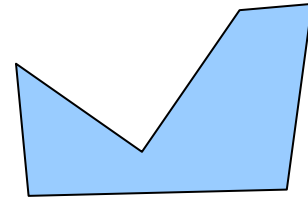
一般多边形裁剪

- 裁剪多边形为凹多边形
- 被裁剪多边形为任意多边形

Weiler-Artherton算法

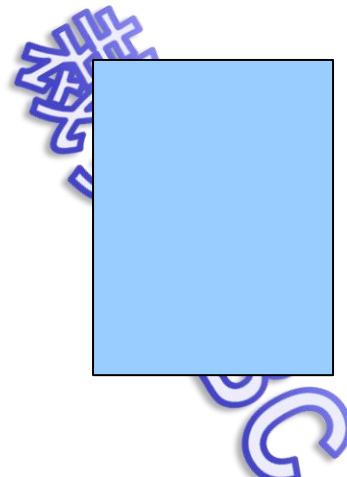
[1] K. Weiler and P. Artherton.
Hidden surface removal using
polygon area sorting. Siggraph
1997, 214-222.

[2] K. Weiler. Polygon comparison
using a graph representation.
Siggraph 1980, 10-18.

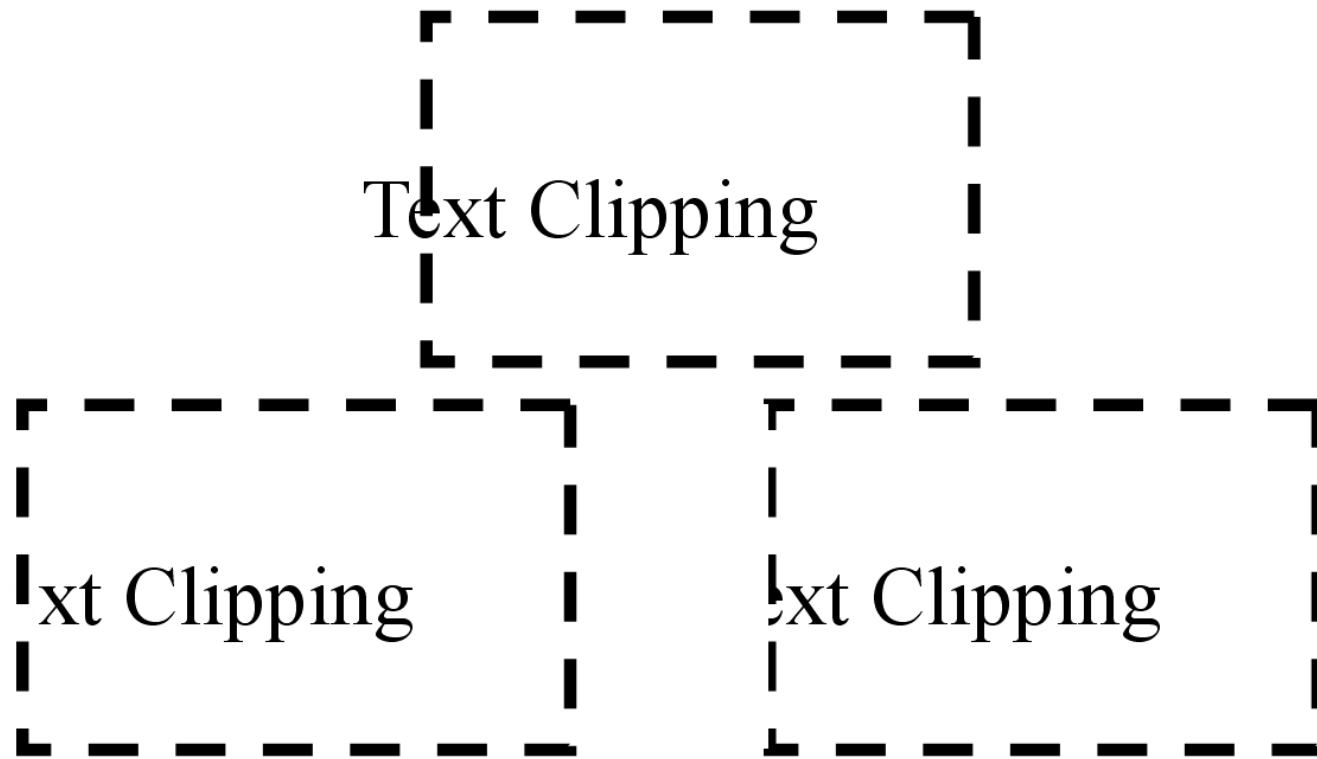


文本裁剪(Text clipping)

- 矢量文本裁剪：采用多边形裁剪算法实现文本的裁剪
- 点阵文本裁剪：
 - 如果点阵是由软件生成的，点阵式文本的裁剪可以归结为点的裁剪问题；
 - 如果点阵式文本由硬件生成，可作简单处理：字符完全位于裁剪窗口内才显示



文本裁剪



文本裁剪

3 三维裁剪

● 用途

- 实体造型中的布尔运算
- 图形流水线中的视见体裁剪
 - 裁剪对象：线裁剪、面裁剪
 - 裁剪窗口：规范的立方体、视域四棱锥



● 裁剪算法

- Sutherland-Cohen、梁-Basky裁剪的推广
- 多边形Sutherland-Hodgman最初即针对三维裁剪

(I. Sutherland and G. Hodgman. Reentrant polygon clipping. Communication of ACM, 1974, 17(1):32-42.)

关于三维变换与裁剪

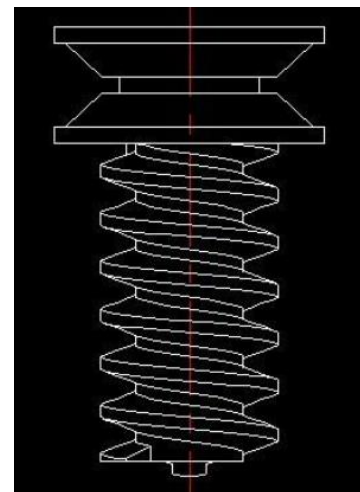
● 何时裁剪？

● 投影前裁剪—三维裁剪

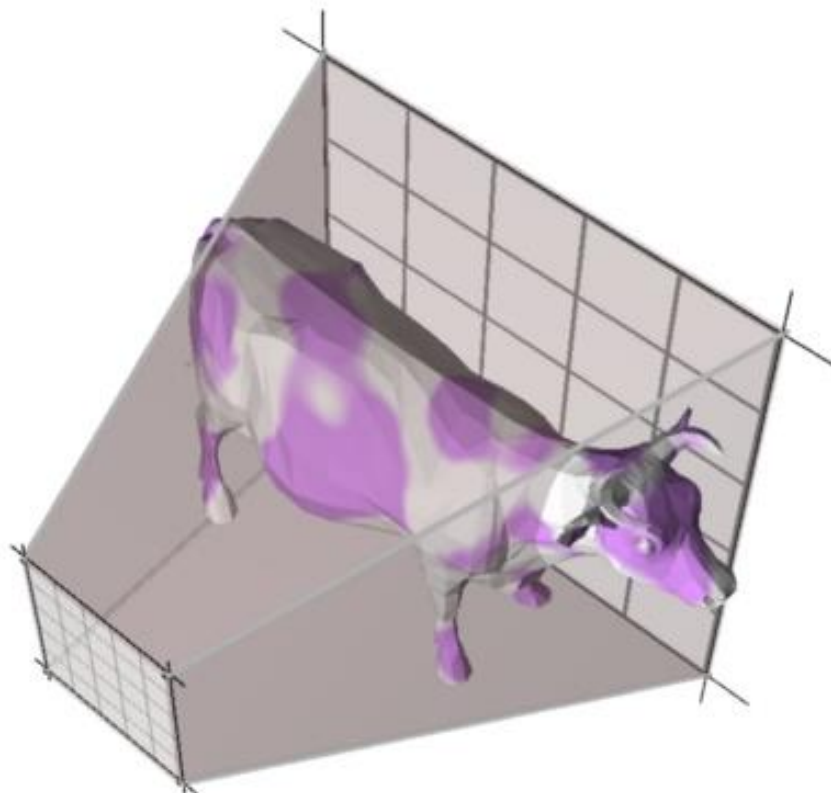
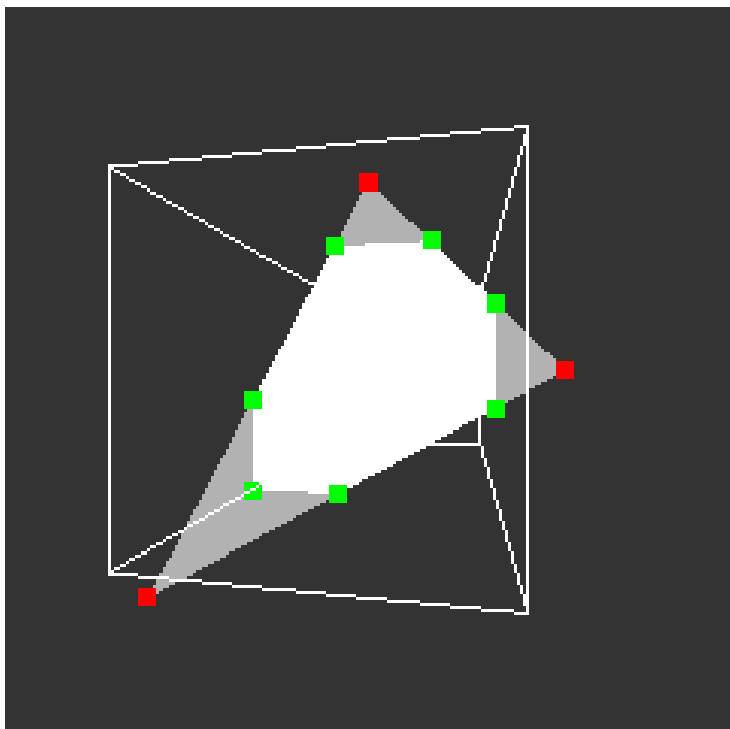
- 优点：只对可见的物体进行投影，提高消隐效率
- 缺点：裁剪实现复杂

● 投影后裁剪—二维裁剪

- 优点：裁剪实现容易
- 缺点：需要对所有的物体进行投影变换



三维裁剪实例



三维裁剪窗口的规范化

- 什么是规范化

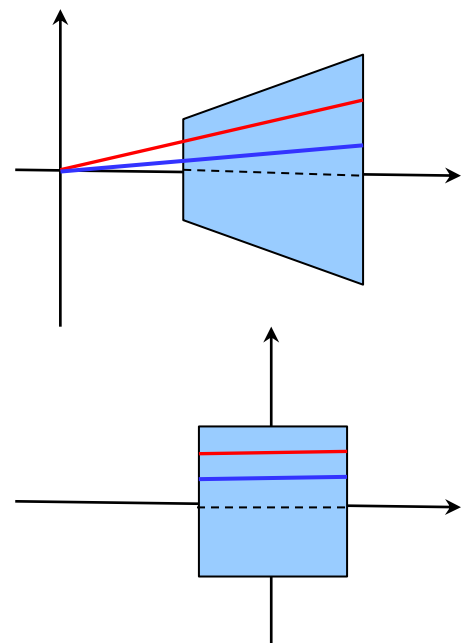
- 梯形 → 正方形 (中心在 origin、边长为 2)
- 视域体 → 立方体 (中心在 origin、边长为 2)

- 为什么引入规范视域体

- 简化投影：将透视投影转为正交投影
- 简化裁剪：棱台裁剪转为长方体裁剪

- 规范化变换

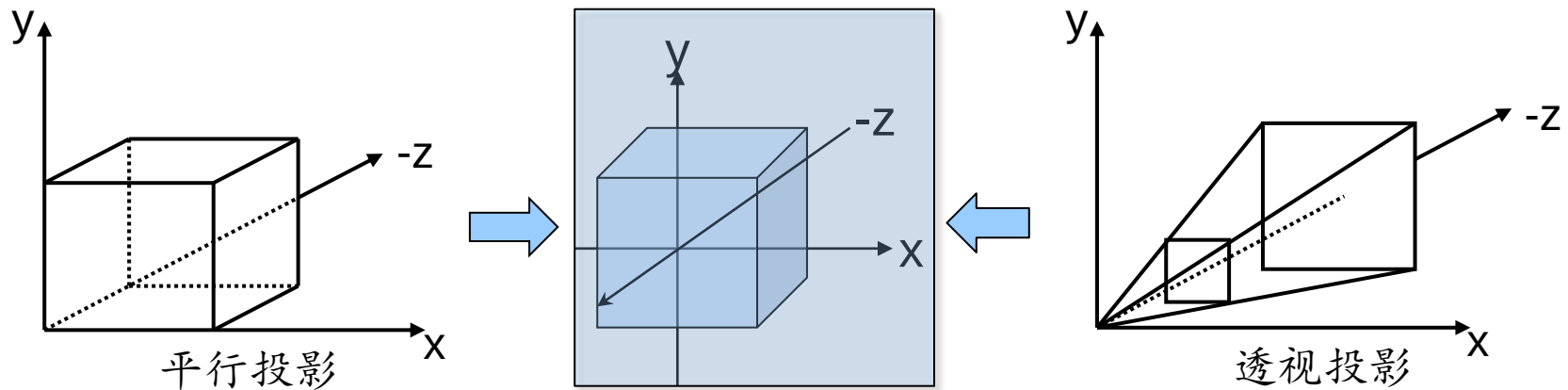
- 实现规范化的变换



三维裁剪窗口的规范化

● 规范化窗口再裁剪

- 平行投影： $[0,1] \times [0,1] \times [-1,0]$ or $[-1,1] \times [-1,1] \times [-1,1]$
- 透视投影： $[-1,1] \times [-1,1] \times [-1,1]$



规范化的视域体

二维情形的规范化

● 梯形→矩形方法

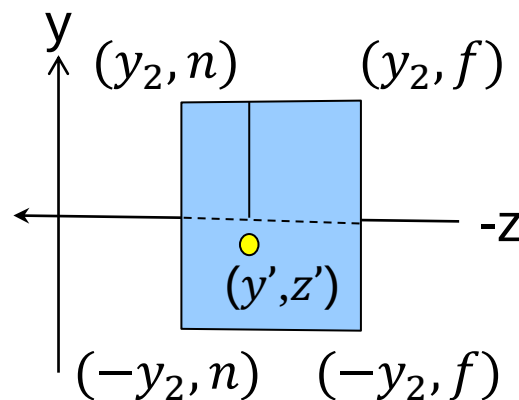
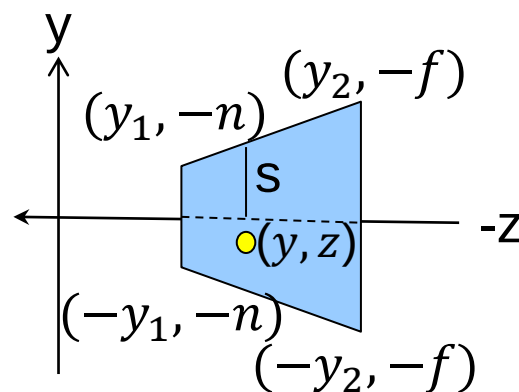
- 梯形腰上的点 → 矩形边上点

$$s = \frac{z+f}{f-n} y_1 + \frac{-n+z}{f-n} y_2 \rightarrow y_2$$

(n :near, f : far)

- 梯形内部点 → 矩形边内部点

$$y' = \frac{y_2}{s} y; \quad z' = z$$



三维规范化(Section 5.7.1)

- 一般情形

- 棱台 $\rightarrow$ 正方体 (右图)

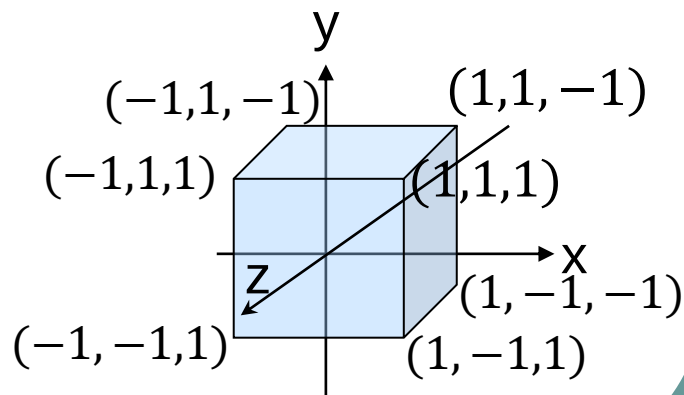
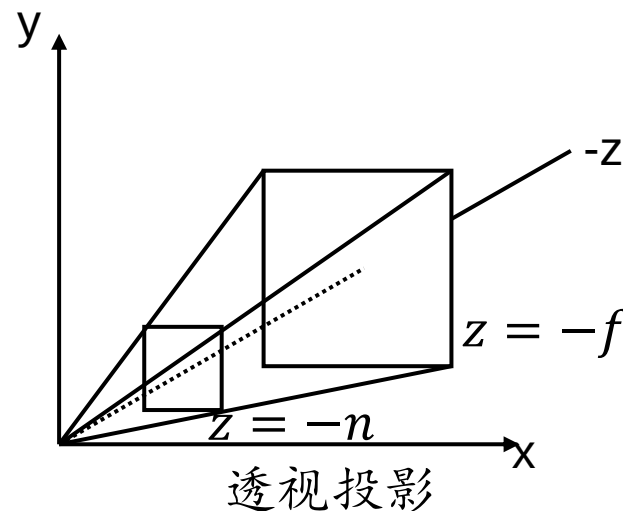
$$z = -f \rightarrow z' = 1; z = -n \rightarrow z' = -1$$

- 张角为 45° 时

$$(x' \ y' \ z' \ 1)^T = N(x \ y \ z \ 1)^T,$$

$$N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$\alpha = -\frac{n+f}{n-f}, \beta = -\frac{2nf}{n-f}$$



总结

- 裁剪
 - 二维线裁剪
 - 二维多边形裁剪
 - 文本裁剪
 - 三维裁剪
 - 关于三维变换与裁剪